

# Case study: compiling ASN.1

Christian Rinderknecht

27 February 2016

# Statement of Purpose

The purpose of this presentation is to convince you that I can handle a C++ frontend and use it to implement whatever static analysis I design that is needed by the embedding application.

Whilst I used to program in C++ quite often ten years ago, I have since become a casual C++ programmer, as this is not a suitable language for writing compilers, if given the choice (especially frontends).

I do not expect this confession to be a serious impediment, though, as I always master a comprehensible subset of each programming language I use, and I fancy myself as a sort of linguist.

## What this could have been about

In 2008 and 2010, Nic Volanschi (Metaware) and I published papers about a new technique we call *unparsed patterns*, and how to use them to extend an existing compiler (here, GCC C++) with user-defined checks. For full details, please read the following:

- ▶ conference version:

<http://home.konkuk.ac.kr/~rinderkn/pub/pepm2008.pdf>

- ▶ journal version (extended):

<http://home.konkuk.ac.kr/~rinderkn/pub/scp2010.pdf>

The reason why I won't be presenting this is that it is a joint work: Nic had the original idea and implemented it on GCC. I contributed by simplifying the algorithms, expressing them in formal logic (also OCaml and Prolog), and proving desirable properties about them.

Of course, this work has everything to do with compilers, but the approach to frontends is unusual: the core idea is to avoid modifying them as much as possible and not parsing at all.

# What this is about

Instead, I will present a past project, which started long time ago, during my Ph.D., and which has become my pet project along these years: compiling ASN.1.

This language is a good case study for a frontend, because it is extremely hard to parse and analyse.

Also, code generation requires a good knowledge of low-level wizardry, down to packing the bits in the frames.

# ASN.1

*Abstract Syntax Notation One* (ASN.1) is a standardised language (ISO, ITU-T) for defining data types whose values may be exchanged across a network between two, possibly heterogeneous peers. It is used in the following contexts.

- ▶ Telecommunication Management Network (SNMP, CMIP);
- ▶ Lightweight Directory Access Protocol (LDAP);
- ▶ email (X.400);
- ▶ UMTS (Radio Access Network);
- ▶ Transferred Account Procedure 3 (billing for GSM/UMTS roaming);
- ▶ 3GPP LTE (Long Term Evolution of GSM/UMTS networks);
- ▶ Multimedia conferencing systems with H.323 (for signalling: H.225, H.235, H.245, H.248, H.450.1);
- ▶ Interactive television services (MHEG);

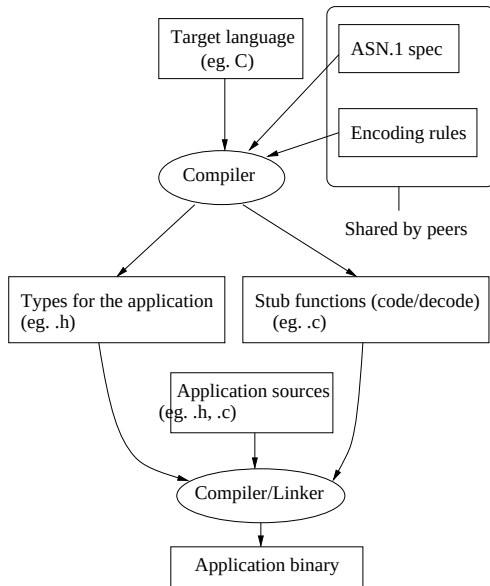
## ASN.1 (usages, continued)

- ▶ Intelligent Transportation Systems (traffic signal control, electronic toll collection, emergency response etc.);
- ▶ Radio Frequency Identification (RFID);
- ▶ Aeronautical Telecommunication Network (air traffic control, flight information services, passenger communications, air-ground protocols etc.);
- ▶ Cryptography for electronic commerce over the internet (SET, PKCS 7, OpenSSL);
- ▶ Smart cards (SIM, WIM and VITALE);
- ▶ National Center for Biotechnology Information (NCBI): specification of millions of DNA sequences;
- ▶ client-server protocol for database access (U.S. Library of Congress: Z39.50);
- ▶ *and many more.*

# Why ASN.1?

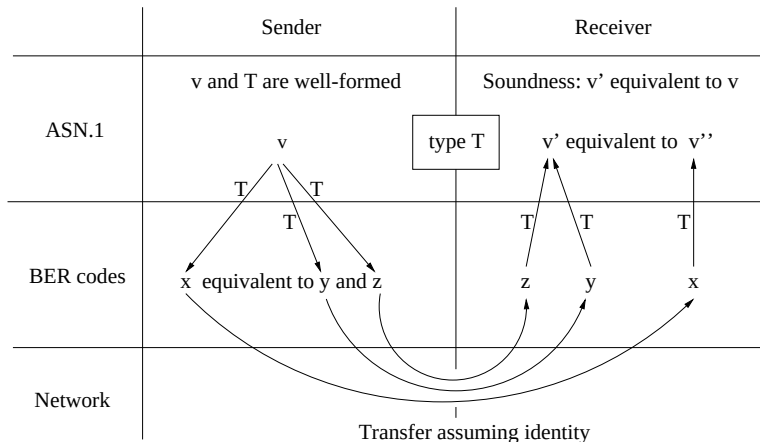
- ▶ It is an abstract language for expressing types and network concepts, independently of any programming language. As a specification language, it is embedded in other languages, e.g., SDL (modelling of protocols with extended finite state machines) and MSC (functional requirements with interaction scenarios), TTCN (test suites) etc.
- ▶ Beyond types, it features network concepts.
- ▶ When used to implement telecommunication protocols, it comes associated with *encoding rules*, which specify how to serialise an ASN.1 value.
- ▶ These encoding rules, like the Packed Encoding Rules (PER) are specially optimised for ASN.1 in order to save bandwidth (although they can be tuned for saving codec time).

# ASN.1 and the compilation chain





# ASN.1/BER and telecommunication



# Issues

1. Specifications: huge (704 pages A4 10pt), complex (concepts mutually dependent).
2. Syntax: large and ambiguous grammar, dynamic extensions.
3. English cannot tell the meaning of all combinations of syntactical constructs.
4. Types denote value sets and type operators denote set operations: subtypes yield general set constraints.
5. Encoding only applies to a subset of ASN.1, defined implicitly and dependent on the encoding rules.
6. Decoding is unspecified.
7. Dynamic checks in codecs (depending on the target programming language) are unspecified.
8. Open source or free compilers: extremely rare, unmaintained, fragile, without precise error reporting.

# Syntax analysis

The BNF of ASN.1 spans 23 pages, versus 19 pages for ISO C++.

When fed to Yacc as it is specified in the standards, it yields between 3,000 and 4,000 shift/reduce and reduce/reduce conflicts (for each kind!). It is ambiguous too.

The situation is actually much worse: many *tokens* are ambiguous, so if the lexer works without any extra knowledge from the parser, there are even more conflicts.

I first solved the problem of the syntax analysis on the 1990 standard, which is smaller, albeit very nasty as well.

# Syntax analysis of ASN.1 1990

Too often engineers expect an off-the-shelf tool that will solve parsing problems. If Yacc is put to work,

- ▶ it may not be suited for precise error reporting (bottom-up parsing, whilst more powerful than top-down, is really bad at providing syntactical context)
- ▶ and understanding conflicts is done on the automaton, hence obscuring the relationship between the conflicts and the grammar (the parsing algorithm must be clearly held in mind at all times, because a conflict may involve several productions and a modification in one of them will change the automaton).

Real ingenuity and fortitude is needed, and I met an engineer who quit after working six months on the syntax of ASN.1 (the company gave up and is still paying an expensive license for a frontend).

## Syntax analysis of ASN.1 1990 (cont.)

Nevertheless, I devised a method and tools that help. It took me three months to obtain an LL(1) grammar of ASN.1 and the corresponding recursive descent parser, which was mathematically proved correct and complete with respect to the standard BNF, and which reported syntax errors very accurately. See

- ▶ in French:

<http://home.konkuk.ac.kr/~rinderkn/pub/RT171.pdf>

- ▶ in English (shorter):

<http://konkuk.ac.kr/~rinderkn/pub/TR171-eng.pdf>

Beyond the academic world, it was employed by some private companies. The case I know best is that of the (ex) Southwestern Bell, where they wanted to check the validity of their U.S. ASN.1 database, used by all the subsidiaries of the group to define their telecommunication protocols.

## Syntax analysis of ASN.1 1990 (cont.)

The programming language used to implement the parser was OCaml, but the open source project Cryptix used the underlying, LL(1) grammar to reimplement my parser in Java. See

<http://cryptix-asn1.sourceforge.net/ASN1-004.grammar>

Cryptix is an early library (still in use) for programming cryptography in Java. (Check the ChangeLog file and search for errors related to “parsing”.)

## Syntax analysis of ASN.1 2008?

I have been working on the latest ASN.1 standards and the current grammar. The increase in size and complexity are still amenable to the same method as the 1990 version, but a summary of the transformations to reach the form LR(1) spans more than 60 pages (versus 45 for the 1990 version). And it is not even LL(1)!

To alleviate the problem, I am writing a tool to check the grammar transformations, which are algebraic in nature because we are dealing with a context-free language. Moreover, if OCaml is used for the semantic actions, I can automatically construct the abstract syntax trees for the final grammar (mapping the algebraic transformation on the concrete syntax onto the abstract syntax).

Nevertheless, future changes to the grammar probably would invalidate an unpredictable amount of previous work, so I have been investigating another approach as well.

## Another approach

The technique I am envisaging now consists in extending the tool used to perform and check the transformations by adding the possibility of *stratifying* the grammar.

This means that we can separate different syntactic layers to reduce parsing conflicts and isolate potential changes.

Moreover, we can type top-level identifiers from the layer of definitions and feed them the lexer *again* (feedback from the parser) so we parse the file multiple times, each time with more typing information and focusing on different parts of the grammar.

But let us move to the static analysis of ASN.1.



# Examples of problems

- ▶ An incorrect specification is rejected too late:

`T ::= T` yields `typedef T T;`  
which is rejected by the C compiler.

- ▶ A correct specification is rejected:

`T ::= SET (WITH COMPONENT (0)`  
`| WITH COMPONENT (1)) OF INTEGER`

- ▶ An incorrect specification is accepted:

`a INTEGER (0<..9) ::= 0`

# Issues in validation

ASN.1 types are considered as sets of values, thus types must have at least one (finite) value.

Because of the great expressivity of ASN.1, the compilers are not likely to fully check arbitrary combinations of subtyping constraints.

- ▶ Vendors' argument: "This is hardly a real problem because those critical specifications are rarely found in practice."
- ▶ My argument: "Maintenance costs are always wildly underestimated. Why not solve completely the problem, get a better product and save money and time?"

# Problems

1. the types may have only infinite values:

$T ::= \text{SET } \{a \ T\}$

→ *this is the finiteness problem;*

2. some value declarations may be ill-typed:

$v \ \text{REAL} ::= ""$

→ *this is the typechecking problem;*

3. especially, some value references may be ill-typed, as

$a \ \text{VisibleString} ::= b$

$b \ \text{INTEGER} ::= 0$

→ *this is the type compatibility problem;*

## Problems (cont.)

4. the subtype constraints may be inconsistent:

`T ::= REAL (SIZE(7))`

→ *this is the consistence problem;*

5. the subtypes may be empty, as

`T ::= SET ((SIZE (1)) ^ (SIZE (2))) OF REAL`

→ *this is the emptiness problem;*

6. the subtypes may have no value set:

`T ::= REAL (ALL EXCEPT T)`

→ *this is the solvability problem.*

# Analysis

- ▶ type compatibility  $\subseteq$  typechecking;
- ▶ constraint consistence, emptiness  $\subseteq$  solvability (the system is solved when we construct explicitly the values of each subtype);
- ▶ typechecking  $\subseteq$  solvability:

$$y \text{ INTEGER } (0..9) ::= 1 \rightarrow \begin{cases} y \ A ::= 1 \\ A ::= \text{INTEGER } (0..9) \\ B ::= A \ (y) \end{cases}$$

where A and B are fresh type references.

So, finiteness and solvability are enough to get a full validation.

## Two approaches for the frontend

Every ASN.1 specification is rewritten in a subset of same expressiveness, with less syntactical constructs. Two strategies:

1. Few preliminary rewrites, followed by a collection of set constraints, then fed to a generic constraint solver.  
<http://home.konkuk.ac.kr/~rinderkn/pub/cj2003.pdf>
2. Lots of preliminary rewrites, simple algorithms to check solvability. (This is like taking the previous method and interleave a dedicated solver with the syntactical rewrites.)  
<http://home.konkuk.ac.kr/~rinderkn/pub/these.pdf>

# Comparison

1. Pros: modularity, less programming. Cons: constraints too general for off-the-shelf solvers, and translation of constraints needed anyway.
2. Pros: easy checking. Cons: eavily specialised, correctness hard to assess.

I need to find a trade-off now.

# The Encoding Rules

The encoding rules also need attention, both in their design and how they are implemented by the code generators or by hand.

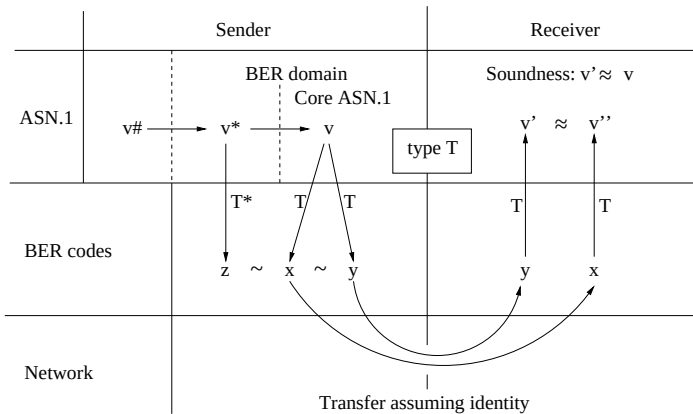
They have been frequently associated with reports of vulnerability.

I showed that these reports were inaccurate insofar as they pointed the finger at the encoding rules, rather than poor implementations produced by backends of ASN.1 compilers. See

<http://home.konkuk.ac.kr/~rinderkn/pub/sam2004.pdf>



# Core ASN.1 and soundness property



# Short presentation of ASN.1

## Some basic types

- ▶ `ok BOOLEAN ::= TRUE`
- ▶ `zero INTEGER ::= 0`  
`DayInTheYear ::= INTEGER {first(1), last(356)}`  
`newYearsEve DayInTheYear ::= last`
- ▶ `SynchroIndicator ::= ENUMERATED {serial,parallel}`  
`synchro SynchroIndicator ::= serial`
- ▶ `pi REAL ::= 3.14159`  
`micron REAL ::= {mantissa 1,base 10,exponent -6}`
- ▶ `byte BIT STRING ::= '0D'H -- or '00001110'B`  
`T ::= BIT STRING {msb(7), lsb(0)}`  
`v T ::= {msb, lsb} -- or '10000001'B`
- ▶ For historical reasons, there are many string types.

## Some constructed types

The SET type corresponds to the record-like structures in programming languages:

```
PersonInfo ::= SET {age INTEGER, married BOOLEAN}  
i PersonInfo ::= {married TRUE, age 42}
```

Some fields can be marked as optional or having a default value:

```
Point ::= SET {x REAL DEFAULT 0, y REAL DEFAULT 0}  
origin Point ::= {}           -- or {x 0.0, y 0.0}
```

## Some constructed types (cont.)

An excerpt from a real protocol:

```
DataAcknowledgementTPDU ::= SET {  
    destRef          Reference,  
    yr-tu-nr         TPDUnumber,  
    checkSum         CheckSum OPTIONAL,  
    subSeqNr         SubSequenceNumber DEFAULT 0,  
    flowControlCnf   FlowControlConfirmation OPTIONAL}
```

The SET OF type corresponds to the mathematical notion of sets with repetition:

```
T ::= SET OF INTEGER  
empty T ::= {}  
small T ::= {7, 9, 1, 1, 3}
```

## Some constructed types (cont.)

The CHOICE type corresponds to a **union** in C.

```
T ::= CHOICE {x REAL, y BOOLEAN}
```

```
u T ::= x : 0.5
```

```
v T ::= y : FALSE
```

The Protocol Data Units (PDU) are CHOICE types, because they model all the possible queries and responses between two peers. A CHOICE type may be recursive, like the other constructed types. One real example (a Network Management Protocol) is:

```
CMISFilter ::= CHOICE {  
    item  FilterItem,  
    and   SET OF CMISFilter,  
    or    SET OF CMISFilter,  
    not   CMISFilter}
```

## Some subtyping constraints

ASN.1 offers a very involved subtyping paradigm consisting of constraints upon recursive types, that restricts their corresponding sets of values in a set-theoretic manner, but also in a structural way.

- ▶ **Interval Constraint:** the INTEGER, REAL and (almost all) string types have totally ordered values, hence allowing interval definitions. For instance:

```
PositiveOrZeroInteger ::= INTEGER (0..MAX)
```

```
PositiveInteger ::= INTEGER (0<..MAX)
```

```
NegativeOrZeroInteger ::= INTEGER (MIN..0)
```

```
NegativeInteger ::= INTEGER (MIN.. $<0$ )
```

```
PositiveReal ::= REAL (0<..PLUS-INFINITY)
```

```
NegativeReal ::= REAL (MINUS-INFINITY.. $<0$ )
```

```
RealInterval ::= REAL (4e-5..1e-4)
```

## Some subtyping constraints (cont.)

- ▶ **Value Constraint:** to restrict the set of values of a type to be a singleton:

```
Wednesday ::= Day (wednesday)
```

- ▶ **Union Constraint:** the new subtype contains the values of the first subtype and of the second subtype (keyword UNION or symbol |):

```
Day ::= ENUMERATED {monday, tuesday, wednesday,  
                    thursday, friday, saturday, sunday}
```

```
WeekEnd ::= Day (saturday | sunday)
```

- ▶ **Alphabet Constraint:** the strings can be restricted to be built upon a given alphabet:

```
CapitalAndSmall ::=
```

```
  IA5String (FROM ("A".."Z" | "a".."z"))
```

```
CapitalOrSmall ::=
```

```
  IA5String (FROM ("A".."Z") | FROM ("a".."z"))
```

## Some subtyping constraints (cont.)

- **Size Constraint:** the values of string types may be constrained to a given sizes, introducing a constraint by the keyword `SIZE`:

```
Exactly31BitsString ::= BIT STRING (SIZE (31))
```

```
StringOf31BitsAtTheMost ::=  
    BIT STRING (SIZE (0..31))
```

```
NonEmptyString ::= BIT STRING (SIZE (1..MAX))
```

The size constraint can also apply to `SET OF` types. In that case, the semantics is very different: the values of the types are sets whose *cardinals* are specified by the size constraint:

```
SetOf5Strings ::=
```

```
    SET (SIZE(5)) OF PrintableString
```

```
SetOfStringsOf5Char ::=
```

```
    SET OF PrintableString (SIZE(5))
```



## Some subtyping constraints (cont.)

- ▶ **Intersection Constraint:** the new subtype contains the values that belong to the two subtypes (keyword INTERSECTION or symbol  $\wedge$ ):

```
FrenchPhoneNumber ::=
  NumericString (FROM ("0".."9")  $\wedge$  SIZE (10))
```

- ▶ **Inclusion Constraint:** to restrict a subtype to have only the values of a given subtype (optional keyword INCLUDES):

```
LongWeekEnd ::=
  Day ((INCLUDES (WeekEnd)) | monday)
Bis ::= Day (WeekEnd | monday)
```

## Some subtyping constraints (cont.)

- ▶ **Complement Constraint:** to restrict the values of a type to *not* belong to another subtype:

```
Lipogramme ::=  
  IA5String (FROM (ALL EXCEPT ("e" | "E")))
```

- ▶ **Constraint on SET OF:** to restrict the elements of a SET OF value:

```
TextBlock ::= SET OF VisibleString  
AddressBlock ::=  
  TextBlock (WITH COMPONENT (SIZE(1..32)))
```

## Some subtyping constraints (cont.)

- ▶ Partial Constraint: to restrict *some* fields of a SET or CHOICE:

```
Quadruple ::= SET {  
    alpha ENUMERATED {in, out} OPTIONAL,  
    beta  IA5String OPTIONAL,  
    gamma SET OF INTEGER,  
    delta BOOLEAN DEFAULT TRUE}
```

we can derive a subtype whose component alpha is always present and equals in, and the component gamma always has five elements:

```
Quadruple1 ::=  
    Quadruple (WITH COMPONENTS {...,  
                                                alpha (in) PRESENT,  
                                                gamma (SIZE (5))})
```

## Some subtyping constraints (cont.)

This subtype has the same values as:

```
Quadruple1 ::= SET {  
  alpha ENUMERATED {in, out} (in),  
  beta  IA5String OPTIONAL,  
  gamma SET SIZE (5) OF INTEGER,  
  delta BOOLEAN DEFAULT TRUE}
```